

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUA – PEC 2

Presentación

Esta PEC 2 y última prueba del curso recoge algunas de las buenas prácticas en el desarrollo y puesta en producción del software en arquitecturas distribuidas. Se trabajan algunos de los conceptos clave durante el desarrollo y puesta en producción, como son la entrega continua, la cultura DevOps y el despliegue utilizando contenedores.

Para elaborar la PEC se formulan cuatro ejercicios: 1) evaluar un aspecto a concretar de los principios fundamentales de la cultura DevOps; 2) razonar sobre los aspectos clave de las pruebas en la entrega continua de software de calidad; 3) evaluar uno de los aspectos clave de los contenedores y plataformas de orquestación; 4) explicar la construcción de un “pipeline” de entrega continua para probar y desplegar los microservicios implementados y testeados a las prácticas anteriores.

La PEC 2 abarca los contenidos de los libros "Microservices Patterns", "Continuous Delivery" y "The DevOps Handbook" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PEC 2 se trabaja las siguientes competencias del Grado en Ingeniería Informática:

- Saber construir aplicaciones informáticas mediante técnicas de desarrollo, integración y reutilización.
- Aplicación de las técnicas específicas de la Ingeniería del Software a las diferentes etapas del ciclo de vida de un proyecto.

Objetivos

Los objetivos de la PEC 2 son:

- Reconocer los factores de entrega, despliegue e integración continua.
- Interpretar la cultura ágil en el desarrollo de software.
- Entender los “pipelines” de entrega, despliegue e integración continua.
- Exponer argumentos sobre las ventajas y costes de los contenedores y plataformas de orquestación.

Descripción de la PEC 2 a realizar

Ejercicio 1 (2,5 puntos)

Al inicio de esta actividad os hemos propuesto el siguiente debate: “Debatir sobre los objetivos básicos que persigue una cultura DevOps y los mecanismos, herramientas y/o estrategias para conseguirlos”. En este ejercicio os pedimos que escojáis uno de los objetivos que hayáis debatido en el espacio “Debate de la Actividad 5” del aula para explicar cómo lo aplicarías al proyecto **Photo&Film4You**.

Solución

En el debate del aula han salido muchos objetivos que persigue la cultura DevOps, entre otros:

- Aumentar la flexibilidad.
- Reducir o eliminar los conflictos entre los departamentos de operaciones y desarrollo.
- Automatizar al máximo los procesos de comunicación entre los departamentos de desarrollo y los de operaciones.
- Agilizar las entregas y la introducción de cambios en producción, consiguiendo una frecuencia de entregas más alta.
- Establecer un sistema de desarrollo y entrega más eficiente que permita reducir costes.
- Reducir los errores en entorno de producción y mejorar así la calidad del producto desarrollado
- Abreviar el ciclo de vida del desarrollo de software
- Proporcionar una entrega continua de software de alta calidad.
- Establecer mecanismos óptimos y fluidos de comunicación entre departamentos para aliviar y transparentar todo lo posible las colaboraciones entre todas las áreas.

Cualquiera de estos objetivos con una explicación clara, concreta, concisa, bien redactada y estructurada, y con un ejemplo de cómo lo haría para conseguirlo en el marco del proyecto **Photo&Film4You**, se considera como solución correcta al ejercicio.

Ejercicio 2 (2,5 puntos)

En la PRAC2 hicisteis la implementación del microservicio *Product Catalog* de **Photo&Film4You** mientras que en la PRAC3 desarrollasteis un conjunto de pruebas unitarias, usando Junit, Spring Boot Test, Mockito y AssertJ sobre este microservicio. Responde brevemente a las siguientes cuestiones:

1. ¿Creéis que desarrollar primero y testear después es una buena estrategia para llevar software de calidad a producción de forma ágil? Razonad la respuesta.

2. ¿Creéis que son importantes estas pruebas para conseguir llevar nuevas funcionalidades a producción de forma rápida y fiable mediante una estrategia de despliegue que implemente Continuous Delivery o, incluso, Continuous Deployment? Razonad la respuesta.

Solución

1. La estrategia adoptada en el proyecto de hacer primero la construcción del software y después la elaboración de las pruebas (tanto unitarias como de integración) **NO** es una buena alternativa para llevar software de calidad a producción de forma ágil. Una mejor alternativa podría ser realizar las pruebas a medida que se van desarrollando las funcionalidades como parte integral del desarrollo de las mismas. Hacerlo así permite tener siempre una buena batería de pruebas ya que estas se pueden considerar como parte integrante del desarrollo de cada funcionalidad.

También se puede ir un paso más allá y desarrollar, para cada funcionalidad, primero los tests que la prueban y después el código que la implementa. Esta técnica es conocida como *Test Driven Development* y es ampliamente utilizada en proyectos ágiles de desarrollo de software.

2. En la PRAC3 se desarrolló toda la parte asociada con el testing de la aplicación (tanto pruebas unitarias como pruebas de integración e incluso pruebas de validación arquitectónica).

Estas pruebas son clave en el desarrollo de software de calidad, concretamente en la rápida entrega de nuevas funcionalidades ya que sin ellas no puedes garantizar que el despliegue continuo de las nuevas funcionalidades se produzca sin errores, por tanto, la parte de testing desarrollada en la PRAC3 **es imprescindible para conseguir tanto Continuous Deployment como Continuous Delivery**.

Como habéis visto, la entrega continua de software se basa en la construcción, prueba y despliegue automatizado de software y uno de los mecanismos para garantizar que el software que se despliega no tenga errores es con una buena batería de test que se ejecuten como parte del *pipeline* de integración continua, que proporcionen una elevada cobertura y aborten la construcción y posterior despliegue en caso de errores. Estos tests no son los únicos que se tienen que pasar, pero sí son una parte importante en el momento de garantizar que las nuevas funcionalidades no introducen errores de regresión o de cualquier otro tipo,

Ejercicio 3 (2,5 puntos)

Elegid un posible patrón de despliegue de entre los cuatro patrones posibles que habéis estudiado para el microservicio *Product Catalog* de **Photo&Film4You**:

- Despliegue con un formato propio dependiente del lenguaje.
- Desplegar el microservicio como una máquina virtual
- Desplegar el microservicio como un contenedor

- Despegar el microservicio como un servicio serverless

Justificad el patrón elegido y enumerad sus ventajas e inconvenientes.

Solución

Un posible patrón de despliegue para el microservicio Product Catalog de Photo&Film4You podría ser el despliegue **usando contenedores Docker y de Kubernetes como plataforma de orquestación**.

Consultar el capítulo “*Chapter 12. Deploying microservices*” de la bibliografía “*Microservices Patterns*”. Concretamente los apartados “*12.3. Deploying services using the Service as a container pattern*” y “*12.4. Deploying the FTGO application with Kubernetes*” para obtener una visión completa de la estrategia y de sus ventajas e inconvenientes.

Ejercicio 4 (2,5 puntos)

Elegid una estrategia de construcción y despliegue a seguir en el proyecto **Photo&Film4You** para conseguir liberar el código desarrollado directamente a producción, automatizando el máximo posible los procesos de test y subida a los entornos. Explicad de forma genérica qué se debería hacer, e indicad como se haría si usamos GitLab como plataforma de CI/CD.

Suponed ahora que los responsables de negocio quieren mantener su estrategia de integración continua y testeo integral de la pipeline para automatizar el despliegue, pero consideran demasiado arriesgado liberar directamente el código a producción sin ninguna aprobación explícita, y prefieren llevar una supervisión de los evolutivos o correctivos incluidos en la versión a desplegar. Elegid la estrategia de construcción y despliegue que se debería seguir para cumplir el requerimiento de los responsables de negocio, y cómo modifica esto los pasos genéricos que habéis indicado para conseguir liberar el código desarrollado directamente a producción.

Nota: No se requiere especificar los comandos concretos de GitLab, solo os pedimos una visión general del proceso y los pasos involucrados en el mismo.

Solución

Para liberar el código desarrollado directamente a producción se debe implementar una estrategia de despliegue **Continuous Deployment**.

Para conseguir *Continuous Deployment* **es necesario que construyamos un pipeline automatizado que permita, de forma muy resumida y simplificada, instalar automáticamente una versión del software a producción cada vez que los desarrolladores integren una nueva funcionalidad en el código fuente**.

Los pasos del *pipeline* a construir, de forma muy general, serían los siguientes:

- Construcción del proyecto. La construcción del proyecto se lanza automáticamente a partir de la subida de nuevas funcionalidades al repositorio de código fuente.
- Ejecución de los distintos tipos de test (unitarios, integración, etc..). El *pipeline* normalmente está estructurado en *Quality Gates* que indican si la *release* puede seguir adelante o se aborta la construcción.
- Despliegue automatizado del software a los diferentes entornos. Para conseguir *Continuous Deployment* es necesario que el software se despliegue automáticamente en cada entorno.

Para implementar un pipeline de *Continuous Deployment* con GitLab se pueden seguir los siguientes pasos:

- Crear un nuevo proyecto en GitLab con su repositorio asociado.
- Definir la política de ramas y merges
- Configurar en el proyecto un runner de CI/CD
- Crear el pipeline que construirá y desplegará el proyecto (`.gitlab-ci.yml`) y que implementará los pasos concretos de construcción, test y despliegue del proyecto.
 - Definir las fases que formaran el pipeline (stages). Estas fases corresponden típicamente con las fases de construcción, test y despliegue.
 - Definir los jobs que se ejecutaran en cada una de las fases definidas anteriormente. Cada job se ejecutará con el “runner” que se le configure en los tags y que deberá estar correctamente configurado y registrado. Cada job se encargará de realizar una tarea concreta dentro de una etapa.

Una vez hecho esto ya se tendría una primera versión del pipeline de *Continuous Deployment* con GitLab.

Podéis encontrar la información completa de estos pasos en https://docs.gitlab.com/ee/ci/quick_start/ y en <https://docs.gitlab.com/ee/ci/introduction/>

La estrategia que se debería seguir para no liberar directamente el código desarrollado a producción se basaría en el **Continuous Delivery** en la que el código se libera automáticamente hasta un entorno previo al de producción, pero el paso a producción no es automático. Se modificaría el último paso del pipeline propuesto para no desplegar automáticamente al entorno de producción.

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson) Chapter 12. Introduction.
- Libro "Microservices Patterns" (Chris Richardson) Chapter 12 "Deploying microservices".

- Libro "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation" (Jez Humble & Dave Farley) Parte I: Chapters 1-10.
- Libro "The DevOps Handbook" (Patrick Debois; John Willis; Jez Humble; Gene Kim): Chapters 1 - 4.

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta misma prueba de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PEC2.pdf*.

Este documento se entregarán en el espacio "Entrega de la PEC2" del aula antes de las **23:59 horas del día 01 de julio de 2024**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación de la PEC 2

	Excelente (10 puntos)	Notable (7 puntos)	Suficiente (5 puntos)	Insuficiente (2,5 puntos)	Sin respuesta / Todo mal (0 puntos)
Ej. 1	Explicación clara, concreta, concisa, bien redactada y estructurada. Ejemplo correcto. (2,5 puntos)	Explicación correcto pero poco concisa y/o mal redactada. Ejemplo correcto. (2 puntos)	Explicación correcto o Ejemplo correcto. (1,5 puntos)	Explicación razonada incorrecto y Ejemplo incorrecto. (0,75 puntos)	Sin respuesta, no hay ninguna justificación. (0 puntos)
Ej. 2	Respuesta 1 correcta y razonada y Respuesta 2 correcta y razonada. (2,5 puntos)	Respuesta 1 correcta pero no (o mal) razonada y Respuesta 2 correcta pero no o mal razonada. (2 puntos)	Respuesta 1 correcta pero no (o mal) razonada o Respuesta 2 correcta pero no o mal razonada. (1,5 puntos)	Respuesta 1 incorrecta con razonamiento y Respuesta 2 incorrecta con razonamiento. (0,75 puntos)	Sin respuesta, no hay ninguna justificación. (0 puntos)
Ej. 3	Patrón muy explicado y Ventajas / inconvenientes correctos. (2,5 puntos)	Patrón muy explicado y Algunas ventajas / inconvenientes incorrectos. (1,5 puntos)	Patrón mal explicado y Algunas ventajas / inconvenientes incorrectos. (1 punto)	Patrón mal explicado y Muchas ventajas / inconvenientes incorrectos. (0,5 puntos)	Sin respuesta, no hay ninguna justificación o todo incorrecto. (0 puntos)
Ej. 4	Estrategia correcta y bien razonada para entregar código a producción sin aprobación. y Estrategia correcta y bien razonada para entregar código a producción con aprobación. (2,5 puntos)	Estrategia correcta pero no muy razonada para entregar código a producción sin aprobación. o Estrategia correcta pero no muy razonada para entregar código a producción con aprobación. (1,5 puntos)	Una de las dos estrategias incorrecta pero razonada, la otra correcta pero con razonamiento incorrecto. (1 punto)	Las dos estrategias incorrectas con razonamiento incorrecto. (0,5 puntos)	Sin respuesta o no hay ninguna justificación. (0 puntos)